

GDN Tree-Scan: Design and Optimization of Tree Verification for Recurrent-Hybrid Language Models

Zhiyuan Ma

zhiyuanma0520@gmail.com

Abstract—Tree speculative decoding for long-running coding agents must preserve coherent continuation across repeated generation and tool calls. We present GDN Tree-Scan, a Gated DeltaNet verifier that connects branch-local computation to accepted-path publication across recurrent, convolution, and attention state. Its implementation combines sequential scan and replay with fixed-shape device acceptance, fused draft selection, captured recurrent replay, and forked FlashAttention-2 with grouped split-K execution. On Qwen3.8-27B NVFP4 on GB10, among runs producing nonempty patches on both shared SWE-bench Verified Astropy tasks, the best recorded tree run achieves 29.09 pooled tokens/s versus SGLang EAGLE’s 26.89, an 8.18% higher rate. In a separate ten-task deployment, all tree-served agent attempts terminated and produced nonempty patches, with no degeneration detected by the recorded trace checks. Component gates check candidate-selection parity and deterministic bounded attention error for the deployed implementation.

Index Terms—speculative decoding, Gated DeltaNet, state commitment, numerical reproducibility, language-model serving

I. INTRODUCTION

Speculative decoding reduces serial target-model invocations by verifying draft tokens in parallel. Its sampling guarantee assumes that verification supplies the intended target probabilities. Tree speculation expands the candidate set, but it does not relax that assumption [1]–[3]. For recurrent-hybrid models, realizing the verifier requires more than an attention mask: a candidate must inherit the state of its ancestors, and the next serving iteration must inherit exactly the path selected for continuation [4], [5].

GDN Tree-Scan turns a candidate token tree into a target-model forward and a native continuation state. Its design connects an ancestry-aware attention path, branch-local GDN computation, a device acceptance walk, and accepted-path publication across recurrent, convolution, and attention key-value (KV) storage. A shared descriptor carries the logical tree through each stage [6], [7]. This system boundary matters because a verifier can compute plausible candidate logits yet publish the wrong state for the next iteration.

The central execution choice is to verify branches in temporary state and replay only the selected path into the native running state. The verifier scans branch paths, while the fixed-shape committer replays accepted operands through native GDN updates in a captured graph. Their finite-precision agreement is a separate requirement, not assumed from the recurrence.

Working revision, September 24, 2026. Current production design and named coding-agent workloads; superseded implementations are archived.

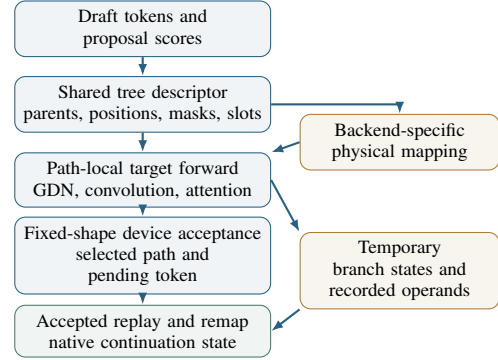


Fig. 1. The tree verifier as a serving pipeline. A shared descriptor connects candidate generation, ancestry-aware target computation, device probability arithmetic, and accepted-path publication. The attention backend determines the physical mapping; all state surfaces must describe the same selected prefix.

This choice exposes concrete optimization questions: how much branch state to cache, when to recompute ancestry, how to organize independent GPU work, and how to keep full-vocabulary probability arithmetic on the device. The attention layout and accepted-state mappings constrain those changes as much as the recurrent equation does.

The paper develops three implementation contributions. *First*, it presents a fixed-shape tree verifier and continuation interface: candidates follow their ancestors during verification, and the selected materialized path becomes the next native state. *Second*, it explains the GPU path-scan and replay design, including transient cut states, shared rounded operands, and publication into request-owned running rows. *Third*, it describes the optimizations integrated into the deployed system: single-logits drafting, fused full-vocabulary candidate selection, spine-first attention layout, grouped-query split-K attention, and device acceptance and captured recurrent replay. The method follows the current fixed-32 Hydra27 implementation. Component gains require an ablation on the same agent workload.

The target workload is a long-running coding agent: generation alternates with repository inspection, tool execution, and tests, and a task can require many model requests. Performance therefore depends on useful completed work as well as token production. Our workload evaluation uses explicitly identified Astropy tasks from SWE-bench Verified [8]. We report the model, serving route, task cohort, and measurement definition together; a decode-rate change is not a task-completion speedup. Component legality checks and loaded-kernel receipts support

the implementation alongside the agent-workload measurements.

Tree verification and compact accepted-state reconstruction have direct prior art. Section II positions our implementation against those mechanisms; we do not claim the first hybrid-tree system or a new GDN recurrence. The architecture and kernel design are developed in Sections III–IV, and Section V separates optimization mechanisms from their measured scope. Measurement provenance is documented in Appendix A; superseded results and local-document timing numbers are excluded from the performance evaluation.

II. BACKGROUND AND RELATED WORK

A. Drafting, verification, and serving

Speculative sampling constructs an exact target sample from proposal and residual distributions, conditional on the probabilities actually supplied [1], [2]. SpecInfer supplies the tree-verification framing; Sequoia, EAGLE-2, and OPT-Tree investigate tree construction and its performance consequences [3], [9]–[11]. Our study keeps these algorithmic questions separate from whether a deployed kernel and cache return the intended probabilities.

PagedAttention and SGLang address the scheduling, memory, and execution substrate [12], [13]. Hybrid serving additionally manages recurrent-state pools alongside KV storage, as documented by SGLang’s hybrid-model integration [14], [15]. Thus neither a fast isolated recurrent kernel nor low transient state memory alone determines user-visible latency. The deployed system uses forked FlashAttention-2 for tree attention, with native-compatible prefill and a specialized fixed-row decode kernel. Its physical layout and work partitioning are part of the verifier design [6], [16].

B. Recurrent and hybrid speculation

Gated DeltaNet augments a decayed recurrent state with a data-dependent delta update [4]. Parallel delta-rule formulations provide relevant algebraic background [17]. Stateful speculation is established territory: STree studies speculative trees for hybrid state-space models, SpecMamba targets an FPGA implementation, and component-aware self-speculation studies how hybrid components affect the design [5], [18], [19]. These works motivate an explicit state lineage; they do not justify treating every recurrent update or numerical implementation as interchangeable.

C. Direct GDN comparisons

Bole derives a tree-structured closed form and evaluates it with a value-tiled finite-polynomial solver. It stores compact update factors and reconstructs the accepted state. Its SGLang integration combines batch-wide verification budgeting with captured GPU execution. Evaluation includes unquantized Qwen3.5-27B on GB10 and online replay of agent sessions [20]. Its state and serving mechanisms directly overlap the problem studied here. The cost of our replay implementation is not a lower bound on alternatives.

TABLE I
DIRECT MECHANISM-COMPARISON SCOPE. THE AUTHORS’
IMPLEMENTATIONS OF THE THREE EXTERNAL METHODS LISTED HERE
WERE NOT EXECUTED IN THIS STUDY.

Method	Overlapping mechanism	Boundary for this study
Bole [20]	Parallel verifier; factorized commit	Different stack and precision; no matched run
TreeWY [21]	Tree solve; accepted-state reconstruction	Different models and operating regime; no matched run
ReplaySSM [22]	Input caching; deferred state work	Alternative state policy; no matched run
GDN Tree-Scan	Scan/replay; accepted-path remapping	Deployed fixed-shape verifier; route-bound checks and agent workloads

Concretely, Bole’s correction system is $(I + G)U = R$, where G contains strict-ancestor interactions and R is the source correction. Tree depth d gives $G^{d+1} = 0$, so $U = \sum_{m=0}^d (-G)^m R$. This finite identity changes the arithmetic schedule from sequential updates to matrix propagation [20]. Our path preserves sequential updates and replays accepted operands; Section V-D explains why the numerical comparison remains an empirical question.

TreeWY expresses GDN tree verification as a triangular system and reconstructs the accepted state from pseudo-values. Its vLLM evaluation uses Qwen3.5-35B and 397B on B200, greedy verification, and disabled prefix caching. It explicitly permits finite-precision disagreement with the baseline and reports that wider trees improve acceptance without yet winning throughput [21]. Its numerical contract and hardware differ from the deployed system studied here.

ReplaySSM caches recurrent inputs and changes when state is materialized; its implementation RFC includes GDN and Mamba2 [22]. Reconstruction and replay are not claimed as new concepts here. Table I states the comparison boundary. Published speed ratios from different checkpoints, precision formats, workloads, and software builds are not combined into a performance ranking.

III. TREE-VERIFIER ARCHITECTURE

A. State and the execution boundary

Let m be a materialized token prefix: the tokens already consumed by the target’s forward computation. Write its persistent state as

$$S(m) = (R(m), C(m), K(m)), \quad (1)$$

where R collects recurrent matrices, C collects convolution histories, and K collects attention KV entries. The logical emitted prefix can also include a newly sampled token b that has not yet been consumed. The continuation boundary is then $(S(m), b)$, not a claim that $S(mb)$ has already been computed. This distinction is essential when counting the correction or bonus token while checking cache state.

A tree descriptor contains candidate token IDs, parent indices, depth/position information, attention visibility, and

maps between logical nodes and physical slots. For a candidate node i , each target layer must consume the state obtained from the candidate’s root-to-parent path. At publication, the selected materialized path, pending-token convention, and next-drafter input row must agree. Figure 2 illustrates the three state surfaces documented by the implementation audit [6], [7].

B. The recurrence and path locality

For one GDN head, let $R \in \mathbb{R}^{d_v \times d_k}$. Queries and keys are in $\mathbb{R}^{d_k}$ and values in $\mathbb{R}^{d_v}$. With decay α_i and write strength β_i , a node applies the standard update [4]

$$R'_i = \alpha_i R_{\pi(i)}, \quad \delta_i = \beta_i (v_i - R'_i k_i), \quad (2)$$

$$R_i = R'_i + \delta_i k_i^\top, \quad o_i = R_i q_i, \quad (3)$$

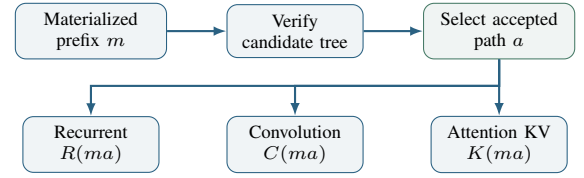
where $\pi(i)$ is its parent. A previous node in the packed buffer need not be $\pi(i)$. The causal convolution likewise requires path predecessors rather than adjacent packed rows. The attention visibility relation must select the same logical prefix and ancestry. These requirements follow from the model’s computation; a correct attention mask cannot repair a wrong recurrent parent.

The implementation verifies nodes with branch-local scratch and makes the selected continuation durable through replay and remapping. This is an implementation route, not the only way to satisfy the contract [6], [7]. In particular, rejecting a branch requires that its contents cannot influence future computation; it does not require zeroing every scratch byte if future reads are correctly restricted.

C. A descriptor shared by verification and commitment

Figure 1 shows the served path. Native multi-token prediction (MTP) supplies the principal draft chain; selected variants add runner-up candidates. The descriptor maps each candidate through the attention mask, recurrent scan, sampler, and selected-path publication. The agent deployment uses Hydra27: 27 active draft nodes excluding the root, packed into 31 physical draft slots plus the root (32 verification rows). Its five-depth MTP head contributes 15 nodes; suffix continuation contributes a six-token principal tail and four- and two-token rescue branches. The maximum draft depth is eleven; four post-root MTP forwards construct the head. Four masked physical slots are padding, not extra logical candidates. Suffix candidates come from the configured Arctic suffix predictor. The current implementation derives the mask, physical packing, acceptance walk, and GDN path schedule from this topology [6].

The FA2 route places the principal spine first in the physical suffix and permutes KV write locations and ancestry-mask key columns while retaining logical query-row and position order. Selection remains in logical-node space; accepted-path publication maps those nodes back to their physical slots. This distinction prevents a layout optimization from silently changing which key/value entries a branch reads. Recurrent paths begin at the native pre-tree state or a transient cut state; the device committer publishes the selected recurrent and convolution state into native running rows and re-linearizes its attention KV entries [6], [7].



Same selected materialized prefix on all three surfaces; correction token b may remain pending for the next forward.

Fig. 2. The accepted-prefix interface. Recurrent, convolution, and attention-KV state must all describe the same materialized prefix. A newly sampled correction or bonus token can remain pending until the next forward.

Algorithm 1 Logical verifier and continuation boundary

Require: Reference-aligned state boundary, candidate descriptor T

- 1: Map logical candidates to physical slots and positions.
 - 2: Verify using path-local convolution, GDN, and attention.
 - 3: Apply the configured sampler to matching target/proposal rows.
 - 4: Obtain the accepted path and any pending correction token.
 - 5: Publish recurrent and convolution state for the materialized path.
 - 6: Map its attention KV entries to the native continuation layout.
 - 7: Map the selected hidden-state row to the next drafter input.
 - 8: Preserve the native pending-token and position convention.
 - 9: **return** Continuation boundary and emitted-token accounting
-

IV. GPU SCAN AND ACCEPTED-PATH REPLAY

A. Path-parallel verification and state lifetime

The deployed GDN verifier decomposes the tree into dependent path groups. A first path starts from the request’s pre-tree state and exports the cut states needed by downstream paths. The second level evaluates independent branch paths from those cut states. Each path applies sequential node updates in ancestry order while separate programs cover value tiles and heads. Candidate outputs continue through the model; rounded operands are recorded for later accepted-path replay [6].

Cut states and operand rings are temporary within a verification step. The durable continuation resides in the request-owned native recurrent and convolution running rows. Replay starts from the pre-tree running state and publishes the accepted frontier back to those rows; prefix-cache snapshots and restores use that native layout. Static work buffers can be reused by CUDA graphs, but they do not replace the request’s persistent state with a branch-leaf pointer. The current lifecycle does not depend on explicitly zeroing every scratch buffer after each step.

This distinction also separates the deployed path-group implementation from an experimental single-launch GDN

Algorithm 2 Logical recurrence and selected-path publication. The implementation groups independent paths into GPU launches and retains only their needed cut states.

Require: Pre-tree state R_0 , descriptor T , rounded node operands x_i

- 1: **for** node i in topological order **do**
- 2: $R_p \leftarrow R_0$ if i has no candidate parent; otherwise $R_p \leftarrow H[\pi(i)]$
- 3: $(H[i], o_i) \leftarrow \text{VerifyStep}(R_p, x_i)$
- 4: **end for**
- 5: Finish target forward and select accepted materialized path A
- 6: $R \leftarrow R_0$
- 7: **for** node i in A , in path order **do**
- 8: $(R, _) \leftarrow \text{NativeUpdate}(R, x_i)$
- 9: **end for**
- 10: Publish R and matching convolution history to native running rows
- 11: Remap accepted KV entries and retire speculative scratch
- 12: **return** Matching continuation state and any pending token

kernel. The latter removes the intermediate cut-state exchange but is disabled in the reported agent deployment. We therefore neither describe the deployed verifier as one GDN launch nor claim that it eliminates all temporary state traffic.

B. Sequential arithmetic and accepted-path replay

Verification uses sequential path scans with fp32 carried state, raw gate inputs, and in-kernel query/key normalization. The deployed fixed-32 committer uses native fused GDN updates with recorded accepted operands and in-place publication to the fp32 state bank. Its graph contains one native update call per GDN layer. Query/key normalization, beta conversion, gate evaluation, and output casts can differ between these implementations; agreement is not guaranteed by their common real-arithmetic recurrence [6].

The accepted path is a device product of the sampling stage. A captured recurrent committer consumes its node indices and lengths and publishes recurrent state. Separate convolution and attention-KV publication consume the same selected path. One graph replay launches the sequence of forty-eight per-layer native updates. It is not a single fused all-layer kernel and does not establish bit identity between verification and publication arithmetic.

C. Temporary state versus recomputation

Path decomposition caches the states needed where downstream paths branch, then reuses them across independent work. If a program exports N_c cut states with value-tile width B_v and key dimension d_k , their fp32 payload is

$$M_{\text{cut}} = 4N_c B_v d_k \text{ bytes.} \quad (4)$$

This is a logical payload, not a register-allocation or total-memory measurement. Recomputing ancestry could remove some exports at the cost of repeating updates. Exporting every

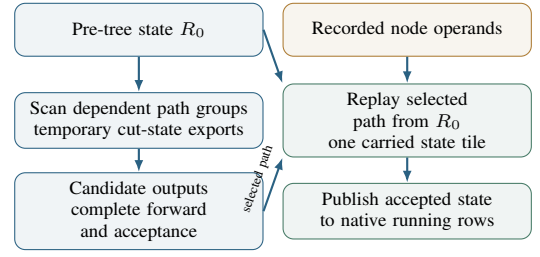


Fig. 3. State lifetime in the deployed path-scan verifier. Temporary cut states connect path groups; recorded operands support selected-path replay. Only the accepted frontier becomes native persistent state.

candidate state would instead trade more storage for less recomputation. The current two-level path schedule occupies a particular point in this design space.

Accepted-path replay carries one state tile through a bounded path loop. Its carried-state footprint is $O(B_v d_k)$ in tree size, while the recorded operands and candidate outputs scale with the verification shape. Static padding makes graph buffers and kernel geometry reusable, but padded rows must remain invisible to attention and selection. Neither a shorter logical path nor fewer state writes alone establishes a faster complete request.

Compact triangular-solve and finite-Neumann methods choose another point in this design space: they form correction factors and reconstruct state without replaying every accepted rank-one update. Their arithmetic schedule differs from sequential scan and replay. Lower state storage or less replay work alone does not establish lower complete-step latency, and a real-arithmetic identity does not settle finite-precision agreement [20], [21].

V. OPTIMIZATION MECHANISMS AND BACKEND CONSTRAINTS

A. Reuse logits and fuse full-vocabulary selection

The deployed drafter uses the full vocabulary. At each head depth, a single logits tensor supplies the principal token and runner-up candidates. The principal chain advances through four captured MTP iterations after the root; sibling proposals reuse the head scores without independent drafter forwards. The suffix predictor extends selected paths within the same fixed verification capacity [6].

A fused CUDA selection kernel emits both the spine token and the three ordered candidate indices directly into static graph buffers. It replaces separate argmax/top-k reductions and their output copies; it does not remove the vocabulary projection itself. Matching the pinned reference requires preserving two tie conventions: argmax chooses the lowest token index among equal maxima, while top-k has its own ordering among selected ties. The fused kernel reproduces both conventions, including duplicate proposals when the reference produces them. Changing that policy would change the proposal tree, so it is a separate algorithmic decision.

B. Keep acceptance and publication on device

The fixed-32 tree-acceptance walk consumes full-vocabulary target/proposal probabilities and produces accepted node indices, emitted tokens, path lengths, and the pending correction or bonus token. It runs through the fixed-shape device route, not the older generic Python node walk. Device-side product filling feeds the captured committer, avoiding host construction of per-layer path lists between selection and publication [6].

Recurrent replay, convolution publication, attention-KV remapping, and the next drafter’s hidden-row selection all consume that same path. CUDA graphs reduce repeated launch orchestration; they do not alter the required selected-prefix contract. Nor does a captured committer imply that every kernel within it has been fused into a single launch.

C. Prepare convolution and replay operands for the fixed shape

Tree convolution gathers a causal window for each candidate from its ancestors and the prior native convolution state. The deployed route uses static path-index preparation, fused tree convolution, and stacked operand rings spanning the GDN layers. Ordered tap operations and cast boundaries remain part of its arithmetic contract. The fixed-32 committer selects the accepted convolution window and writes it to the native running row; graph scratch is prepared for the next iteration rather than becoming durable per-node history [6].

D. Spine-first layout and grouped split-K attention

The forked FA2 kernel applies tree visibility inside attention. Its spine-first packing keeps the principal path contiguous in the candidate suffix. This matters numerically as well as logically: inserting masked branch columns between spine keys can change the reduction layout of an online softmax. Permuting KV slots and the matching ancestry-mask key columns preserves logical query order while controlling the physical key layout [6].

The deployed GQA-pair kernel shares staged key/value tiles across two query heads. Split-K divides the context walk into four partitions. Each partition computes a local maximum, denominator, and weighted-value sum; a combine kernel rescales and merges these partial results. This exposes more independent work at batch one, but changes floating-point accumulation order. Its qualification therefore uses explicit determinism and numerical bounds against both the incumbent and a high-precision reference, rather than asserting byte identity with the incumbent.

The workload receipt identifies the specialized kernel in all sixteen full-attention layers and records the binary hash, four-way split, graph identity, and no-fallback dispatch. Prefill uses the patched FA2 integration’s native-compatible route. These execution records establish which implementation generated the workload observations; performance attribution still requires a matched component ablation.

E. Publish one coherent native continuation

If accepted logical node a_j becomes continuation position j , the KV remapper copies from the physical slot assigned

to a_j into the request’s native destination slot for j . The slot permutation is applied to verification reads and writes as well as publication. A valid ancestry mask alone cannot repair inconsistent addressing.

Recurrent and convolution publication make the same selected frontier visible in native running rows. Prefix caching and full/piecewise CUDA graphs are enabled in the reported deployment. Their state ownership follows the native request lifecycle; temporary cut-state exports remain a distinct within-step resource.

VI. EVALUATION SETUP

Performance is evaluated on coding-agent workloads, with the task subset and serving configuration attached to every reported rate. The Qwen3.8-27B NVFP4 evidence uses SWE-bench Verified Astropy tasks on a single GB10/DGX Spark: a shared two-task comparison of three tree runs against SGLang EAGLE, plus a separate ten-task tree deployment [6], [8]. The best recorded shared-task rate supplies the headline; later runs and the larger deployment remain in the evaluation. These selected subsets are not the full benchmark. Every decode-rate result uses the pooled completed-request estimator in Eq. 5, excluding first-output latency. Recorded agent duration includes the agent loop, model service, tools, and repository operations; server startup and subsequent evaluator time are separate.

The system description follows the latest deployed Hydra27 route supported by these records. Component validation is tied to the fused-selection and split-K artifacts used by that route. Archived experiments on superseded implementations do not supply results or qualification claims for the current system. Newer topology or kernel candidates without a completed serving record are likewise excluded from the deployed-method description.

VII. RESULTS

A. SWE-bench Verified coding-agent workload

We evaluate recorded deployments on Astropy instances from the test split of `princeton-nlp/SWE-bench_Verified`, using Qwen Code 0.19.4 to inspect repositories, edit files, execute tools, and produce patches. A shared two-task comparison covers three tree runs and one SGLang EAGLE run that each produced nonempty patches on both tasks. A separate ten-task segment, Cqc10, recorded on 24 August 2026, supplies the latest complete single-configuration deployment case. The task/evaluation worker is x86-64; model serving runs on the GB10. These are full agent attempts. Table III lists every task and saved evaluator outcome in Cqc10.

a) *Ten-task deployment:* The served model is the port’s `qwen3.8-27b-nvfp4-radixark` checkpoint with Mod-eOpt mixed quantization and bfloat16 execution dtype. The route uses Hydra27 with 32 physical verification rows, full-vocabulary draft logits, draft-logit reuse, and a patched FA2 GQA-pair split-K4 attention kernel. Prefix caching and full/piecewise CUDA graphs are enabled. The engine permits one active sequence, with maximum context 131,072

TABLE II

OPTIMIZATIONS IN THE RECORDED CURRENT DEPLOYMENT. WORK REMOVED IS A MECHANISM DESCRIPTION; NO COMPONENT TIMING IS CONVERTED INTO AN AGENT-WORKLOAD SPEEDUP.

Mechanism	Work addressed	Deployed realization	Evidence boundary
Single-logits drafting	Repeated projection and score preparation	Full vocabulary; four captured MTP iterations after root	Source and boot engagement
Fused top-3 selection	Separate argmax/top-k reductions and copies	One CUDA selection into graph buffers	Byte and graph-replay component gates
Path-parallel GDN scan	Serial branch work and repeated ancestry	Two-level path schedule with transient cut states	Source-selected route; single-launch candidate disabled
Device acceptance and commit	Host path construction and launch orchestration	Fixed-32 walk, device products, captured committer	Boot and route receipts; no blanket sampler-law proof
Convolution and operand preparation	Repeated path-index and per-layer preparation	Fused convolution, pregather, stacked layer rings	Enabled/engaged deployment path
Spine layout and KV remap	Inconsistent physical ancestry and continuation	Spine-first permutation plus accepted-path remap	Common slot mapping in verification/publication
FA2 grouping and split-K	Repeated KV staging and insufficient independent work	Query-head pairs; four context partitions and combine	Numerical credential matches the loaded binary
Native-state lifecycle	Persistent branch-specific cache ownership	Publish native running rows; graphs and prefix cache enabled	Current state contract and recorded agent behavior

TABLE III

ALL TEN TASKS IN THE COMPLETED CQC10 ASTROPY CASE. IDS HAVE PREFIX `ASTROPY__ASTROPY-`. TIMES ARE AGENT ELAPSED MINUTES, ROUNDED FROM SAVED RUNNER METADATA; EVALUATION IS SUBSEQUENT. THIS SELECTED DEVELOPMENT SEGMENT HAS ONE ATTEMPT PER LISTED TASK AND NO MATCHED NATIVE ARM.

Task suffix	Agent time (min)	Saved evaluation
13977	24.59	Failed tests
14096	22.99	Resolved
14182	7.23	Failed tests
14309	1.67	Resolved
14365	6.78	Failed tests
14369	27.70	Resolved
14508	41.24	Resolved
14539	18.57	Resolved
14598	25.79	Failed tests
14995	4.59	Resolved

tokens and a 4,096-token scheduler budget. Source commit `e7af6b595a8b` and the loaded extension hash are retained in the artifact. The implementation uses the graph/cache-enabled stochastic route described above.

The proxy supplies temperature 0.6, top- p 0.95, top- k 20, minimum- p 0, and presence penalty 1.0; the engine seed is zero. The per-response output cap is 24,000 tokens, with a separate 20,000-token compaction cap and a 9,000-second agent timeout. Proxy auto-continuation is disabled. The agent runs in the task’s repository image on an internal network that permits the model proxy and blocks external egress; per-task records retain the verified network-policy fingerprint. Agent duration comes from the task runner’s elapsed-time field: it includes the agent loop and tool work, and excludes server startup and subsequent evaluation. We do not infer a completed-task speedup from model-only counters.

b) Workload observations: All ten attempts have evaluator reports: six resolved their tasks and four failed tests. Recorded agent durations sum to 181.15 minutes, with a range of 1.67–41.24 minutes. This is an observed task ledger, not a

representative SWE-bench Verified success estimate. The sum is accumulated agent time, not elapsed campaign time including startup and evaluation.

Every task used tools and produced a nonempty patch. A source-bound replay of the recorded trace checks detects no degeneration in these ten runs and no malformed tool-call arguments. The degeneration heuristic jointly requires at most 200 visible characters, zero tool calls, and at least 2,000 thinking characters; none of the ten traces meets that conjunction. This screen can miss loops that continue narrating or using tools, so its zero detections establish neither universal degeneration freedom nor preserved task quality. The earlier campaign’s detected degeneration remains an adverse observation in the lineage audit.

c) Pooled decode comparison: For completed request r , let n_r be its output-token count, ℓ_r its server-recorded end-to-end latency, and f_r its time to first output. With R completed requests and $N = \sum_r n_r$, we use

$$D_{\text{pool}} = \frac{\sum_{r=1}^R (n_r - 1)}{\sum_{r=1}^R (\ell_r - f_r)} = \frac{N - R}{\sum_r \ell_r - \sum_r f_r}. \quad (5)$$

Counts and durations are pooled before division. The numerator uses the conventional $n_r - 1$ output intervals; outputs arriving in one batch can have zero arrival gaps. The denominator excludes first-output latency and between-request agent/tool work. It is accumulated request time, which counts overlapping requests separately, not isolated GPU time or elapsed campaign time.

For Cqc10, $N = 248,342$, $R = 265$, $\sum_r \ell_r = 10,667.6211$ seconds, and $\sum_r f_r = 989.8239$ seconds yield 25.63 tokens/s. The 265 requests reconcile with 256 normal requests and nine successful internal compaction requests; all recorded completion reasons are stop. The ten pre/post metric brackets are contiguous. Generation-token and completed-request counters agree with their latency-histogram populations, with no counter reset or outstanding request at the selected boundaries.

The eligible SGLang EAGLE comparator, recorded on 24 August, serves the as-shipped Qwen3.8-27B NVFP4 checkpoint with FlashInfer, using speculative steps/top- k /draft-token settings of 3/1/4. It uses the same Qwen Code version, agent harness, requested sampling settings, and network policy, with a 24,000-token response cap, a 262,144-token context limit, and an 8,192-token prefill chunk. Its two completed Astropy tasks are 12907 (resolved; 7.10 agent minutes) and 13033 (failed tests; 26.96 agent minutes). Their counters give $N = 47,854$, $R = 45$, $\sum_r \ell_r = 2,013.6344$ seconds, and $\sum_r f_r = 235.9488$ seconds, yielding 26.89 tokens/s. Both brackets contain streaming requests only. All completed engine requests are included, including internal agent traffic.

d) Performance eligibility: After inspecting the archived runs, we restrict the shared-task rate comparison to run-pairs in which both tasks completed and produced nonempty patches. We apply this retrospective rule to both systems and retain patches that fail tests. All three tree pairs and one of the two recovered SGLang pairs meet the rule. The other SGLang pair is excluded: task 13033 produced an empty patch after a final thinking-only response at the configured output cap, so the test harness was not invoked. Its failed outcome and original records remain in the artifact. We exclude the entire pair from this performance comparison, rather than retaining only its successful task. The rates therefore characterize patch-producing runs, not performance over all attempted tasks.

e) Best recorded shared-task result: We select task IDs 12907 and 13033 because both have completed SGLang records, then report all three recovered split-K tree runs on that pair. Sr12 and Cqc16 ran on 19 August; Cqc15 ran on 23 August. Their engagement receipts identify Hydra27 with 27 valid draft nodes, maximum draft depth 11, 32 physical verification rows, full-vocabulary drafting, and GQA-pair split-K4 on all sixteen full-attention layers, with no fallback. They share the loaded attention extension but differ in Python patch revisions. Each uses a 32,768-token response cap, compared with the eligible SGLang run’s 24,000. All 41 Sr12 responses and all 45 eligible SGLang responses are at most 20,000 tokens, below both configured caps; no response uses the differing output allowance. All Sr12 requests terminate with stop.

The best shared-task tree observation is Sr12: $N = 25,027$, $R = 41$, $\sum_r \ell_r = 989.2612$ seconds, and $\sum_r f_r = 130.4422$ seconds give **29.09 tokens/s**, 8.18% above the eligible SGLang rate under Eq. 5. Later observations are 28.28 for Cqc16 and 27.27 for Cqc15; Table IV retains them alongside the best result. All selected task brackets pass the same population and idle-boundary checks. Every row on the shared pair has one resolved task and one failed task. All six selected tree attempts used tools and produced nonempty patches, with no detected degeneration or malformed tool-call arguments under the recorded checks. Every selected tree request finished with stop, without a recorded response-cap hit.

The tree headline is its best recorded rate on the shared pair, not the latest run or Sr12’s entire workload. Sr12 completed four tasks in total, two resolved and two failed; pooling all four gives 26.21 tokens/s. Cqc16’s subsequent task 13236

TABLE IV
POOLED DECODE RATES FOR TREE VERIFICATION AND SGLANG EAGLE WITH QWEN3.8-27B NVFP4 ON GB10, USING EQ. 5. THE FIRST FOUR ROWS USE THE SAME TWO SWE-BENCH VERIFIED ASTROPY TASKS, 12907 AND 13033; EACH INCLUDED RUN PRODUCES NONEMPTY PATCHES ON BOTH TASKS; DECODER SETTINGS ARE SPECIFIED IN THE TEXT. CQC10 COVERS A SEPARATE TEN-TASK SUBSET. BOLD MARKS THE BEST RECORDED ELIGIBLE TREE RATE.

Deployment	Tasks	Requests	Tokens/s
SGLang EAGLE	2	45	26.89
Tree Sr12 (Aug 19)	2	41	29.09
Tree Cqc16 (Aug 19)	2	35	28.28
Tree Cqc15 (Aug 23)	2	53	27.27
Tree Cqc10 (Aug 24)	10	265	25.63

exhibited degeneration, and Cqc15’s subsequent task 13398 lacks a completed evaluation. Those adverse records remain in the artifact and outside this explicitly two-task comparison. The shared-task comparison evaluates the complete decoding systems, including their proposal topology, draft budget, and serving optimizations. These method-specific choices are part of the comparison. The records use the same requested sampling settings and agent harness. The performance table retains every eligible run-pair: three tree observations and one SGLang observation. This retrospective subset does not establish performance over all attempts or each system’s optimum over draft lengths and candidate budgets. Repeated runs with a predeclared tuning protocol would measure that performance envelope; a component ablation would separately isolate the contribution of tree verification. The artifact retains the original metric brackets, source-bound configuration and behavior records, and reproducible pooled reductions.

f) Selection and comparison limits: Cqc10 contains the ten remaining tasks of an interrupted sixteen-task development campaign. Earlier segments changed source revisions and output caps and included degeneration, incomplete output, and budget-capped attempts. Source-bound audit records of those failures, retries, and missing evaluations remain in the artifact; the original records remain at their recorded source paths. We do not relabel the completed segment as the entire campaign, combine those segments into a single-configuration benchmark score, or drop unsuccessful attempts from a claimed benchmark denominator. The tasks were selected from one repository, and this deployment has one recorded boot without a matched ten-task native run. Separate targeted native probes change other serving settings and do not supply that comparator. Consequently, these records demonstrate an implemented verifier serving real coding-agent work, but establish no native-versus-tree task-speed or quality advantage. A current, matched task campaign is the remaining experiment for that claim.

g) Existing comparator experiments: Earlier SGLang EAGLE trials and native MTP controls remain in the repository. The earlier native campaign predates the NVFP4 deployment; later NVFP4 native probes cover targeted tasks with different serving settings. DSpark measurements use synthetic requests, and a later DFlash2 check establishes boot and response

handling rather than agent-task performance. These records are distinguished from the pooled SWE comparison above; their superseded or non-agent rates are not additional performance results for the current verifier.

B. Validation of the deployed components

The fused full-vocabulary selection gate compares the two output tensors against the pinned argmax and top-k reference operations. Across 1,368 input cases and five block-count settings (6,840 configurations), it records zero byte mismatches; planted negative controls fire. A separate captured four-level selection graph completes 24 replays with zero mismatches. These results validate candidate-selection outputs for the tested inputs and graph behavior, not end-to-end task equivalence [6].

The split-K numerical credential is bound to the same FA2 binary and patch identity recorded by the ten-task deployment. It reports bit-identical repeated output and log-sum-exp values within each of sixteen determinism cases, with matching digests across two processes. On synthetic query/key/value tensors drawn at scales measured from banked model operands, 93.31% of output elements agree with the incumbent within two ULP, the maximum absolute output difference is 0.00390625, and maximum log-sum-exp disagreement is four ULP. All nine declared checks pass, including comparisons against a float64 dense reference and checks for non-finite disagreements. These are attention-component bounds; split-K changes rounding and is not claimed to be byte-identical to the incumbent or to preserve every downstream token.

Loaded engagement is checked separately from component eligibility. The deployment receipt records the specialized FA2 arm in all sixteen attention layers, four splits, and no fallback, while the boot records identify fused selection, fixed-32 device acceptance, and captured state publication. The agent traces and evaluator outcomes in Section VII-A supply the complementary application evidence. Neither passing component gates nor observing no degeneration in a finite task cohort establishes a full-model conditional-distribution proof.

VIII. DISCUSSION AND REMAINING EVIDENCE

The deployed design couples tree geometry, physical layout, and continuation publication. Optimizing any one of these in isolation can move work elsewhere or invalidate another stage’s mapping. The current implementation uses transient cut states, static graph buffers, and native persistent rows for different purposes; conflating them would obscure both memory cost and state ownership.

Sequential replay and compact reconstruction remain alternative implementation choices. The present paper does not benchmark the authors’ Bole or TreeWY systems, and its deployed-component gates do not rank those methods. A direct comparison requires compatible models, matched workloads, and each system’s actual execution path.

For long-running agents, task completion and task wall time are the relevant application outcomes. More decoded tokens can reflect additional reasoning or a longer tool loop. The pooled decode comparison is therefore accompanied by task

identities, outcomes, agent/tool policy, and terminal budgets. It describes the evaluated deployments and the stated retrospective patch-producing subset; it is not a task-completion speedup estimate.

Current-component legality and whole-task performance answer different questions. The selection gate checks exact candidate outputs; the split-K gate checks deterministic bounded rounding; the workload tests exercise the composed system. A broader quality claim requires held-out tasks and repetitions. Isolating an optimization’s application benefit requires toggling that component within the same eligible serving configuration and task protocol.

IX. CONCLUSION

GDN Tree-Scan implements tree verification for a recurrent-hybrid model by connecting an ancestry-aware target forward to accepted-path state publication. A shared descriptor coordinates attention visibility, branch-local GDN computation, convolution history, device probability arithmetic, and the next native continuation. Sequential scan and accepted-path replay make the state lifetime and arithmetic choices explicit; cached state, recomputation, physical layout, and independent-work batching expose the main optimization tradeoffs.

On two shared SWE-bench Verified Astropy tasks with Qwen3.8-27B NVFP4 on GB10, among runs producing nonempty patches on both tasks, the best recorded tree run attains 29.09 pooled tokens/s versus SGLang EAGLE’s 26.89, an 8.18% higher rate. Later tree runs on the same pair attain 28.28 and 27.27; the separate ten-task deployment attains 25.63. All ten attempts in that larger deployment terminated with nonempty patches and no degeneration detected by the recorded trace checks. The common rate definition compares the complete decoding systems; draft budgets, selection scope, and task outcomes are reported alongside the rates.

The contribution is a concrete verifier design and its implementation mechanisms, supported by current-component checks and workload-specific observations. A broader performance claim requires additional tasks and repeated runs with a declared tuning protocol. The present results do not establish broader task-quality preservation, full-model sequential equivalence, or general superiority over SGLang or the published compact-state systems.

APPENDIX A

REPRODUCIBILITY AND CLAIM BOUNDARIES

The artifact records source revisions, effective serving settings, component binary identities, topology, graph/cache configuration, and original task metadata. The latest deployed design is determined by promotion history and positive execution receipts, not by the date of an experiment that happened to revisit an older route. Superseded implementations and their diagnostic results remain available as dated audit records; they are not presented as the current method.

All workload rates use the completed-request pooled decode estimator in Eq. 5. Token totals and latency sums come from the same completed-request population. First-output latency,

tool execution, and evaluator time are separate quantities. Earlier global-token/subset-wall proxies and local-document timing do not supply paper performance claims. The selection ledger preserves every included and excluded run-pair and the unchanged unfiltered measurements.

Algebraic correctness concerns the real-arithmetic recurrence. *Numerical agreement* concerns particular kernels and references. *Sampler correctness* concerns the acceptance/residual law for the supplied probabilities. *Serving correctness* additionally requires coherent state across continuation and cache operations [2], [6], [17]. Equal candidate outputs, bounded attention error, and observed task completion address different parts of that chain.

For target probabilities p and proposal probabilities q , ordinary rejection sampling combines accepted mass $\min(p, q)$ with residual mass proportional to $(p - q)_+$. This gives the intended target marginal only for the probabilities actually supplied [1], [2]. A correct abstract sampling rule cannot compensate for stale state or different target logits. The paper therefore does not promote component checks or task outcomes into a general distribution-preservation claim.

APPENDIX B

REMAINING WORKLOAD COMPARISONS

A matched current native-MTP/tree comparison would freeze the checkpoint, task cohort, agent harness, sampling, tool policy, and budgets, then report outcomes and completion time over every attempted task. Each method may use its own speculative depth and serving optimizations; the tuning protocol and selected configuration must be explicit. Confirmation should predeclare performance eligibility and retain unsuccessful attempts in the outcome report.

Further component ablations, concurrent-agent experiments, and Bole/TreeWY system comparisons are conditional additions to that protocol. They are not inferred from archived snippet timing, unserved kernel candidates, or experiments on superseded routes.

REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” 2022. [Online]. Available: <https://arxiv.org/abs/2211.17192>
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating Large Language Model Decoding with Speculative Sampling,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.01318>
- [3] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, “Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification,” 2024. [Online]. Available: <https://arxiv.org/abs/2305.09781>
- [4] S. Yang, J. Kautz, and A. Hatamizadeh, “Gated delta networks: Improving mamba2 with delta rule,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.06464>
- [5] Y. Wu, Z. Qin, A. Wong, and S. Soatto, “Stree: Speculative tree decoding for hybrid state-space models,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.14969>
- [6] Z. Ma, “Lumo FlyWheel implementation and experiment archive,” https://github.com/MaCoredroid/Lumo_FlyWheel/tree/984f613d7885d9bf7898866827552b68a3199d0c, 2026, revision evidence snapshot; individual campaigns retain their own run identities.

- [7] —, “Stateless Tree Speculative Decoding: A Branched Verifier That Caches and Commits Like Native,” https://macoredroid.github.io/Lumo_FlyWheel/stateless-tree.html, 2026, author experiment archive; accessed 2026-09-21.
- [8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” 2023. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [9] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, “Sequoia: Scalable, robust, and hardware-aware speculative decoding,” 2025. [Online]. Available: <https://arxiv.org/abs/2402.12374>
- [10] Y. Li, F. Wei, C. Zhang, and H. Zhang, “Eagle-2: Faster inference of language models with dynamic draft trees,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.16858>
- [11] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, “Opt-tree: Speculative decoding with adaptive draft tree structure,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.17276>
- [12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [13] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient Execution of Structured Language Model Programs,” 2023. [Online]. Available: <https://arxiv.org/abs/2312.07104>
- [14] SGLang Team, “Hybrid Models Meet SGLang: More than Full Attention,” <https://pytorch.org/blog/hybrid-models-meet-sglang-more-than-full-attention/>, 2025.
- [15] SGLang Project, “Qwen3-Next SGLang documentation,” <https://lmsys.org/mintlify/app/cookbook/autoregressive/Qwen/Qwen3-Next>, 2026.
- [16] T. Dao, “Flashattention-2: Faster attention with better parallelism and work partitioning,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.08691>
- [17] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, “Parallelizing Linear Transformers with the Delta Rule over Sequence Length,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.06484>
- [18] L. Zhong, S. Xu, H. Wen, T. Xie, Q. Guo, Y. Wang, and M. Li, “Specmamba: Accelerating mamba inference on fpga with speculative decoding,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.19873>
- [19] H. Borobia, E. Seguí-Mas, and G. Tormo-Carbó, “Component-aware self-speculative decoding in hybrid language models,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.01106>
- [20] L. Wang, Y. Su, X. Wu, C. You, Y. Liu, Z. Qiu, J. Zhang, J. Zheng, F. Liu, J. Zhang, C. Tian, and C. Huan, “Bole: Efficient Tree Speculation for Hybrid-Attention Language Models,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.01651>
- [21] S. M. Ghantasala, “TreeWY: Speculative Verification for Gated DeltaNet Hybrids,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.20961>
- [22] Dao AI Lab and NVIDIA, “ReplaySSM: Cache SSM Inputs Instead of State,” <https://github.com/vllm-project/vllm/issues/47572>, 2026, implementation RFC; accessed 2026-09-21.